

INTRODUCTION

Efficient Charge Recovery Logic (ECRL) is proposed as a candidate for low-energy adiabatic logic. Power comparison with other logic circuits is performed on an inverter chain and a carry look ahead adder (CLA). ECRL CLA is designed as a pipelined structure for obtaining the same throughput as a conventional static CMOS CLA. Interest in low energy computing is growing as the cost of power dissipation becomes preposterous. Adiabatic computing, which uses an AC power supply rather than DC, is an attractive approach in this view point. Most of the proposed adiabatic circuits use diodes or diode-like devices for pre charge. This causes unavoidable energy loss due to the voltage drop across the diodes. However, energy dissipation as a whole is still smaller than that of static logic. ECRL adopts a new method that performs pre charge and evaluation simultaneously. ECIU eliminates the pre charge diode and dissipates less energy than other adiabatic circuits. An ECRL inverter chain and a pipelined CLA are constructed to show the effectiveness of this approach.

Power consumption plays a major role in present day VLSI design technology. The demand for low power consuming devices is increasing rapidly and the adiabatic logic style is said to be an attractive solution. This paper investigates different methods for designing adiabatic logics such as CMOS adiabatic circuits, ECRL, PFAL, CAL, DFAL, 2N-2P and 2N-2N2P. A lot of research has been already done using adiabatic technique. This paper presents a review of above mentioned adiabatic inverters in literature from basic elements. The SPICE simulation results show the comparison of power dissipation between conventional CMOS circuits and adiabatic logic based circuits.

1.1 ORGANISATION OF PROJECT

This project consists of 7 chapters and each chapter consists the following: Chapter1: This chapter gives the introduction of the project.

Chapter2: This chapter gives a brief introduction about ECRL circuit.

Chapter3: This chapter describes the design of full adder

Chapter4: This chapter explains the tanner tools

Chapter5: This chapter gives a brief introduction about designing on tanner tool

Chapter6: This chapter gives output waveform

Chapter7: This chapter gives the conclusion and future work for this project.

2. ECRL and its characteristics

2.1 INTRODUCTION

Interest in low energy computing is growing as the cost of power dissipation becomes preposterous. Adiabatic computing, which uses an AC power supply rather than DC, is an attractive approach in this viewpoint. Previous works on adiabatic circuits show their potential for real applications. Most of the proposed adiabatic circuits use diodes or diode-like devices for pre charge. This causes unavoidable energy loss due to the voltage drop across the diodes. However, energy dissipation as a whole is still smaller than that of static logic. ECRL adopts a new method that performs pre charge and evaluation simultaneously. ECIU eliminates the pre charge diode and dissipates less energy than other adiabatic circuits. An ECRL inverter chain and a pipelined CLA are constructed to show the effectiveness of this approach.

2.2 ENERGY COMPARISON IN ECRL CIRCUITS

Previous adiabatic circuits need diodes for precharging output nodes. Using diode for precharge leads to an unavoidable energy loss due to a voltage drop across the diode when turned on. Thus, energy loss in the previous works is given as

$$E_{loss} \approx C V_{DD} \Delta V.$$

ECRL always pumps charge on the output with a full swing. However, as the voltage on the supply clock approaches to, the recovery path to the supply clock is disconnected, thus, resulting in incomplete recovery. Amount of loss is given as

$$E_{loss} \approx C K^2 / 2.$$

Although non-zero, it is still 5-10 times smaller than the previously mentioned adiabatic circuits. Fig. 4 shows the comparison of energy dissipation with static CMOS and 2N-2N2D with varying data rate. CMOS technology with the threshold voltage of 0.2V is used in simulation. An inverter chain is used for simulation, and energy is

calculated per transition with the same input. Energy dissipation in the static inverter chain does not vary over transition frequency. ECRL dissipates less energy than other logic family especially with small load capacitance in a low frequency operation. Some points not drawn on the graph at high frequency represent mal-function due to incomplete

transition. As shown in Fig. 5, computing energy is reduced for all types of logic as the supply voltage decreases. ECRL maintains a relative energy gain at scaled supply voltages. Energy dissipation varies as the threshold voltage changes. Lowering the threshold voltage helps the adiabatic circuits as is already shown by the energy loss equations (1) and (2). In contrast to adiabatic circuits, a static circuit consumes

slightly more energy because of the increased crossbar current. The simulation results of the inverter chain for the effect of different threshold voltages indicate this trend as shown in fig. A plot of energy per addition for ECRL CLA is drawn in Fig.5. The relative energy gain is significant with the same throughput as a static CMOS. This result shows the beneficial use of ECRL in real applications. ECRL reduces the power dissipation of the 16-b CLA by about 4-6 times over a

static CLA in various supply voltages.

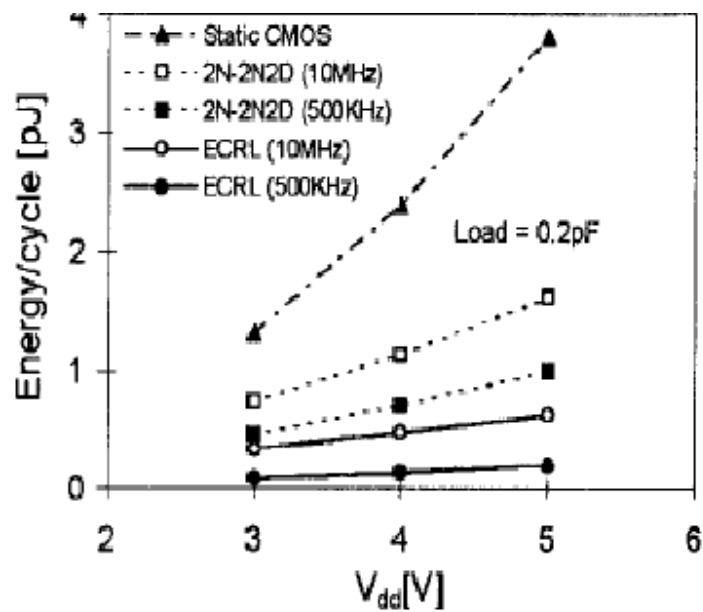


Fig. 5. Energy vs. supply voltage

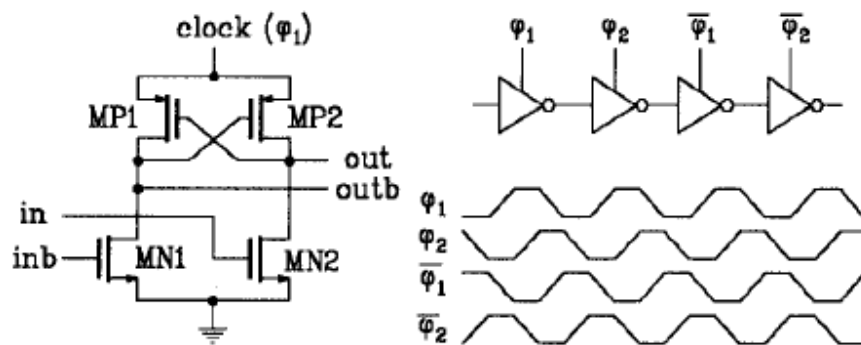


Fig. 1. Schematic of ECRL inverter and supply clock

3. ADABATIC LOGIC

Adiabatic circuits are **low power** circuits which use "reversible logic" to conserve energy.^[1]

Unlike traditional **CMOS** circuits, which dissipate energy during switching, adiabatic circuits reduce dissipation by following two key rules:

- Never turn on a **transistor** when there is a voltage potential between the **source** and **drain**.
- Never turn off a transistor when current is flowing through it.

3.1 HISTORY

"Adiabatic" is a term of Greek origin that has spent most of its history associated with classical **thermodynamics**. It refers to a system in which a transition occurs without energy (usually in the form of heat) being either lost to or gained from the system. In the context of electronic systems, rather than heat, electronic charge is preserved. Thus, an ideal adiabatic circuit would operate without the loss or gain of electronic charge.

The first usage of the term "adiabatic" in the context of circuitry appears to be traceable back to a paper presented in 1992 at the Second Workshop on Physics and Computation. Although an earlier suggestion of the possibility of **energy recovery** was made by **Charles H. Bennett** where in relation to the energy used to perform computation, he stated "This energy could in principle be saved and reused".

Etymology of the term "adiabatic logic". Because of the **second law of thermodynamics**, it is not possible to completely convert energy into useful work. However, the term "adiabatic logic" is used to describe logic families that could theoretically operate without losses. The term "quasi-adiabatic logic" is used to describe logic that operates with a lower power than static CMOS logic, but which still has some theoretical non-adiabatic losses. In both cases, the nomenclature is used to indicate that these systems are capable of operating with substantially less power dissipation than traditional static CMOS circuits.

3.3 PRINCIPLES

There are several important principles that are shared by all of these low-power adiabatic systems. These include only turning switches on when there is no potential difference across them, only turning switches off when no current is flowing through them, and using a power supply that is capable of recovering or recycling energy in the form of electric charge. To achieve this, in general, the power supplies of adiabatic logic circuits have used constant current charging (or an approximation thereto), in contrast to more traditional non-adiabatic systems that have generally used constant voltage charging from a fixed-voltage power supply.

3.4 POWER SUPPLY

The power supplies of adiabatic logic circuits have also used circuit elements capable of storing energy. This is often done using inductors, which store the energy by converting it to **magnetic flux**. There are a number of synonyms that have been used by other authors to refer to adiabatic logic type systems, these include: "charge recovery logic", "charge recycling logic", "clock-powered logic", "energy recovery logic" and "energy recycling logic". Because of the

reversibility requirements for a system to be fully adiabatic, most of these synonyms actually refer to, and can be used interchangeably, to describe quasi-adiabatic systems. These terms are succinct and self-explanatory, so the only term that warrants further explanation is "clock-powered logic". This has been used because many adiabatic circuits use a combined power supply and clock, or a "power-clock". This is a variable, usually multi-phase, power-supply which controls the operation of the logic by supplying energy to it, and subsequently recovering energy from it.

Because high-Q inductors are not available in CMOS, inductors must be off-chip, so adiabatic switching with inductors are limited to designs which use only a few inductors. Quasi-adiabatic stepwise charging avoids inductors entirely by storing recovered energy in capacitors.^{[2][3]} Stepwise charging (SWC) can use on-chip capacitors.^{[4]:26}

Asynchratic logic, introduced in 2004,^{[4]:51} is a CMOS logic family design style using internal stepwise charging that attempts to combine the low-power benefits of the seemingly contradictory ideas of "clock-powered logic" (adiabatic circuits) and "circuits without clocks" (asynchronous circuits).

3.5 CMOS Adiabatic circuits

There are some classical approaches to reduce the dynamic power such as reducing supply voltage, decreasing physical capacitance and reducing switching activity. These techniques are not fit enough to meet today's power requirement. However, most research has focused on building adiabatic logic, which is a promising design for low power applications.

Adiabatic logic works with the concept of switching activities which reduces the power by giving stored energy back to the supply. Thus, the term adiabatic logic is used in low-power VLSI circuits which implements reversible logic. In this, the main design changes are focused in power clock which plays the vital role in the principle of operation. Each phase of the power clock gives user to achieve the two major design rules for the adiabatic circuit design.

- Never turn on a transistor if there is a voltage across it ($V_{DS} > 0$)
- Never turn off a transistor if there is a current through it ($I_{DS} \neq 0$)
- Never pass current through a diode

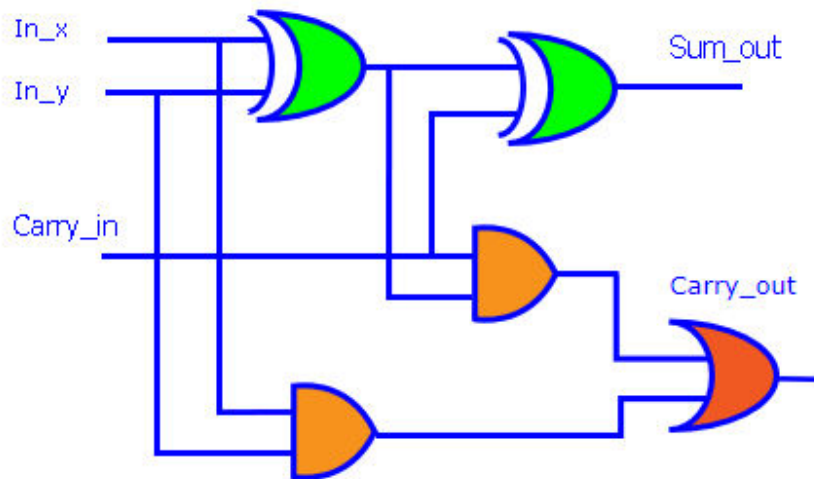
If these conditions with regard to the inputs, in all the four phases of power clock, recovery phase will restore the energy to the power clock, resulting considerable energy saving. Yet some complexities in adiabatic logic design perpetuate. Two such complexities, for instance, are circuit implementation for time-varying power sources needs to be done and computational implementation by low overhead circuit structures needs to be followed.

There are two big challenges of energy recovering circuits; first, slowness in terms of today's standards, second it requires ~50% of more area than conventional CMOS, and simple circuit designs get complicated.

4. ABOUT FULL ADDER

4.1 INTRODUCTION

Full-adder is a digital circuit to perform arithmetic sum of two bits and a previous carry. It is represented in the diagram and truth table below.



Full Adder Truth table (FA) below:

In_x	In_y	Carry_in	sum_out	Carry_out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum_out= [(in_x) XOR (in_y)] XOR [(carry_in)]

Carry_out = [(in_x) AND (in_y)] OR [(in_x XOR in_y) AND carry_in]

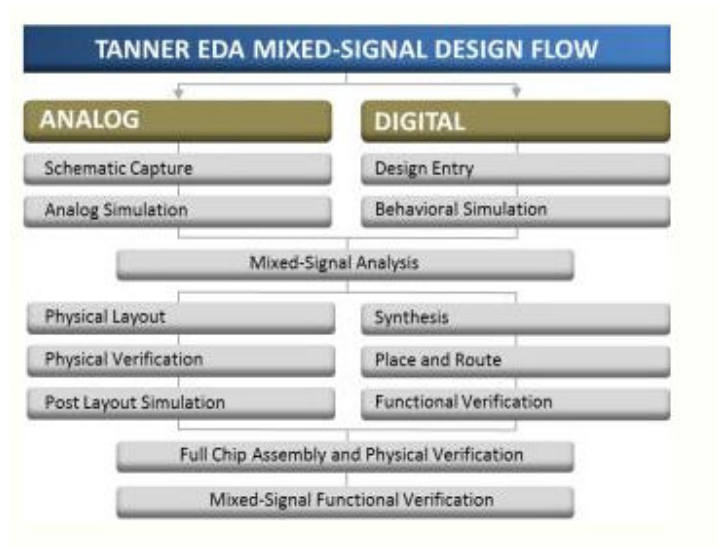
5. TANNER TOOLS

5.1 INTRODUCTION

Tanner EDA by Mentor Graphics now enjoys the stability that comes with joining a successful big 3 EDA company. Mentor will continue to invest in the Tanner EDA suite to strengthen it as the user base and support Tanner EDA products, their customers and their foundry partners and place particular focus on IoT users who design and integrate: power electronics, sensors, MEMS, photonics and RF circuitry.

Tanner EDA solutions by Mentor Graphics provides a complete line of EDA software tools that drive innovation for the design, layout, and verification to tape-out of analog and digital

integrated circuits (ICs). Tanner EDA solutions offer a complete design environment supporting analog, mixed-signal, or MEMS domains in one highly-integrated end-to-end flow. Our customers are creating breakthrough applications in areas such as Automotive, Life Sciences, imaging, sensors, power management, Military, Aerospace, Space, Internet of Things (IoT) and ARM Powered® solutions. Tanner EDA products have a low learning curve, high interoperability, and a powerful user interface improve design team productivity.



3.2 WORKING

Tanner EDA is a suite of tools for the design of integrated circuits. These tools allow you to enter schematics, perform SPICE simulations, do physical design (i.e., chip layout), and perform design rule checks (DRC) and layout versus schematic (LVS) checks. There are 3 tools that are used for this process: S-edit - a schematic capture tool T-SPICE - the SPICE simulation engine integrated with S-edit L-edit - the physical design tool

3.2.1 Using S-Edit (Schematic Entry Tool) & T-SPICE (Analog Simulation Tool)

S-edit is a schematic entry tool that is used to document circuits that can be driven forward into a layout of an integrated circuit. It also provides the ability to perform SPICE simulations of the circuits using a simulation engine called T-SPICE. T-SPICE can be setup and invoked from within S-edit.

Part 1: Setup your Directory Structure & download Libraries

- a) Log onto a computer on 6th floor Cobleigh.
- b) You want to create a directory for all of your Tanner EDA projects. You also will need to download and unzip a set of library & model files from the course website that will be used for your simulations. - Create a directory structure named "EELE414_VLSI_Fall2011\Tanner_Projects"

c) Go to the course website and download the zip file called “Tanner_Libraries.zip”. Unzip it into your Tanner Projects directory. This group of files contain the necessary information to enter components into S-edit (circuit symbols), perform SPICE simulations (models), and do physical layout (layer definitions, DRC, LVS)

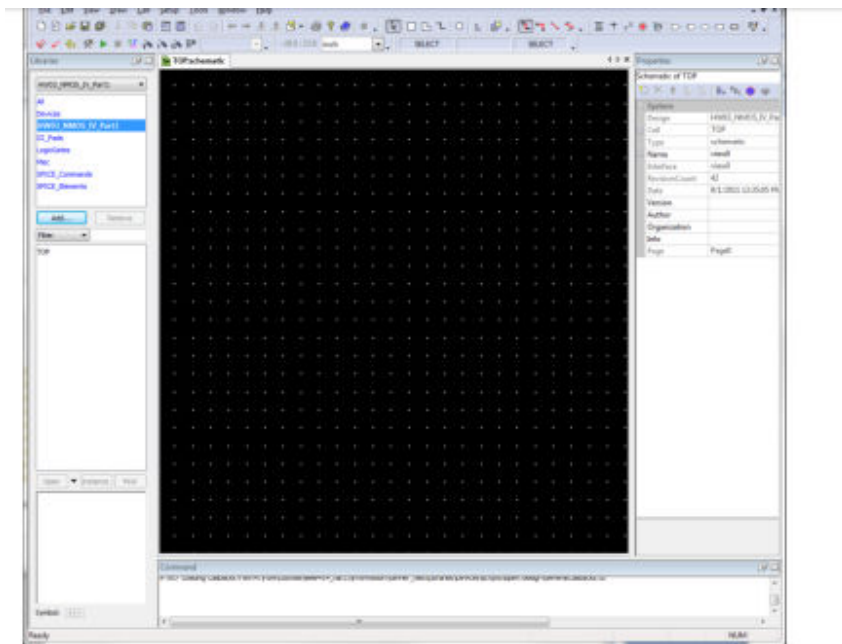
Part 2: Start a New Design & Setup Libraries a) Start S-Edit: - Start – All Programs – Tanner EDA – Tanner Tools v12.6 – S-Edit v12.6 b) Start a New Design: Using the pull down menus, create a new design: - File – New - New Design A dialog will appear asking for a design name and location. When you give the name, Sedit will create a folder of that name in the directory that you provide that will contain all of the design files. You should give a descriptive name that represents each simulation you will be running.

- Enter the name “HW03_NMOS_IV_Part1” and browse to your “EELE414_VLSI_Fall2011\Tanner_Projects” directory - Click “OK” c) Create a new Cell A “cell” is a design element. A cell can contain multiple views such as schematics and 3 symbols. Cells can be instantiated in other cells. When performing a simulation, we will typically call the cell “TOP”. When we are testing a circuit, for example an inverter, the inverter will have its own cell that contains a schematic of the devices and a symbol. The inverter cell is instantiated in the TOP cell that contains ideal elements such as voltage sources and probes that are only used for simulation.

This allows us to separate the cells that are actually going to be implemented on the die versus cells that are only used for simulation. Using the pull down menus, create a new cell view:

- Cell – New View: - enter the cell name “TOP”. Ensure the design name is “HW03_NMOS_IV_Part1” and click OK. You can leave the interface and view names “view0”. A blank schematic page will appear. It is a good idea to save this right now.

d) Enter the symbol libraries: First, you need to include a library which contains the symbols for all basic circuit elements such as resistors, NMOS, capacitors, etc... The libraries for all the basic symbols are in the Tanner_Libraries.zip file you downloaded and unzipped. - On the left side of the S-edit screen you’ll see a Libraries window, click on the “Add” button. - Browse to “Libraries\All\All.tanner” and click “OK” You should see a set of libraries appear:



e) Setup the SPICE Models for the Generic_025 kit. The libraries that you just added have symbols for NMOS and PMOS transistors. However, all non-linear components such as MOS transistors require a model to describe their behavior.

If you simply enter an NMOS symbol in your schematic, SPICE will not know what to do since each NMOS transistor fabricated in a different technology behave differently. In this example, we will use a transistor technology called “Generic_025”, which represents a standard, 0.25um CMOS process.

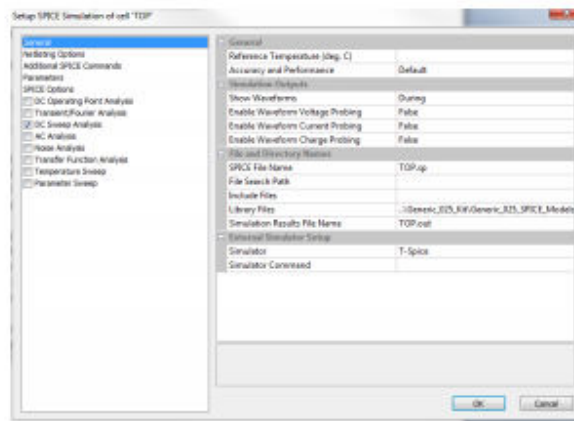
You will need to setup the SPICE models for this process in S-edit. Once you do that, when you enter an NMOS or PMOS transistor, you can then associate the 0.25um model to that symbol. Using the pull down menus, setup the SPICE models: - Setup – SPICE Simulation - In the dialog that appears, you should highlight “General” on the left. - On the right, click in the “Library Files” field. This is where you will specify any SPICE models you will be using in your simulations.

Browse and select “Generic_025_Kit\Generic_025_SPICE_Models_Level1.lib”

- On the right, click in the “SPICE File Name” field. This is where you specify the name and location of the SPICE Netlist output. Browse to your design directory “EELE414_VLSI_Fall2011\Tanner_Projects\HW03_NMOS_IV_Part1” and enter the filename “TOP.sp”.

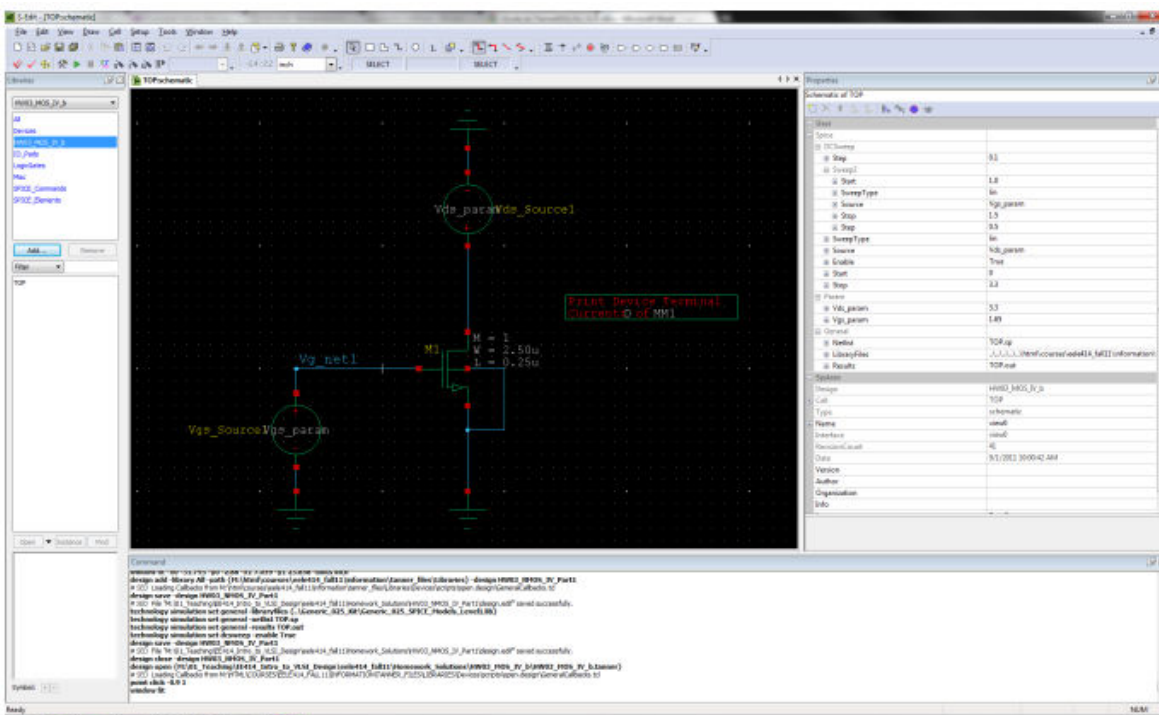
- On the right, click in the “Simulations Results File Name” field. This is where the results of the simulation will be written. This file is what the waveform viewer will look for when you go to plot your results. Browse to your design directory

“EEL414_VLSI_Fall2011\Tanner_Projects\HW03_NMOS_IV_Part1” and enter the filename “TOP.out”. - Before you can exit this window, you will need to select an analysis type. We will setup the details of the analysis later, but for now, just check the “DC Sweep Analysis” and click



“OK” to close the setup window.

Part 3: Enter the Schematic to simulate the IV behavior of an NMOS Transistor We will be entering the following circuit.

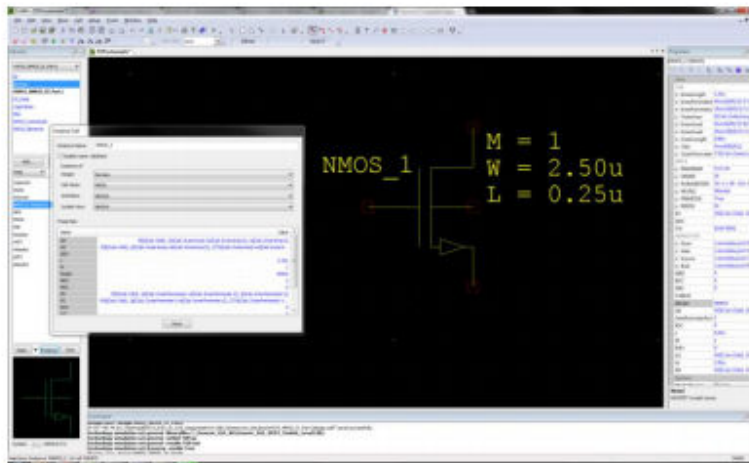


a) Enter the NMOS transistor

- On the left, click on “Devices” in the upper window. This will display all of the symbols available in this group. You should see all of the components that you can implement on a CMOS integrated circuit.

- On the bottom left window, click once on “NMOS”. You should see the symbol of the NMOS transistor show up in the symbol viewer window at the bottom.

- To place the NMOS, you will click on the “Instance” button. Two things happen when you click on this button. First, a dialog will appear that will allow you to setup the parameters for the NMOS. Second, the symbol will attach to your mouse. We will place the NMOS in the schematic first and then set its properties later. This is an easier way to enter the device. Click in the schematic window to drop an instance of the NMOS. Hit the “Esc” button to end the insert-mode.



The NMOS is now in the schematic. A note on zooming: - [Home] = zoom fit - [-] = zoom out - [=] = zoom in - the scroll wheel also zooms in/out.

- To setup the NMOS, click on the NMOS symbol. You will see the properties of the device on the left. We want to setup the following:

- Model : enter “NMOS”. This model is found in the Generic_025 library you added - Name: M1. The SPICE designation for MOS transistors is to have the name start with an “M”. S-edit automatically appends an M to the name is the final name will be “MM1” in the TOP.sp file.

But it is good practice to name all MOS transistors with M’s. - W Set to 2.5u. This is the default. - L Set to 0.25u. This is the default. b) Enter a DC source for VGS

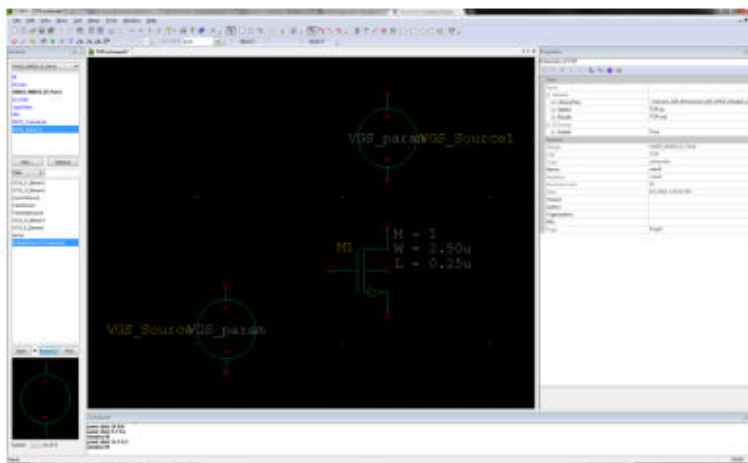
- Using the same process you used for the NMOS symbol, enter a “SPICE_Elements:VoltageSource”. This is a generic voltage source symbol that is configured as a DC, TRAN, PWL, etc.. in its properties dialog.

- Click on the voltage source and enter the following: - MasterInterface: DC (this is the default but this is how you would change it to something else.

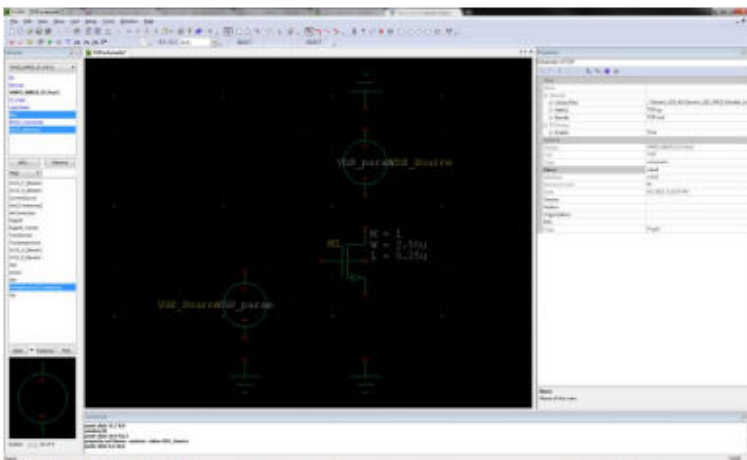
- Name: VGS_Source (it is a good idea to use descriptive names) 7 - V This is where you will set the DC voltage (i.e., 4v, 5v). However, for this example we will use a parameter instead of a hardcoded value. We will enter a parameter name here and then set up the parameter later. Enter “VGS_param” for the value of V. When performing a DC sweep, you must use parameters for the sweep.

c) Enter a DC source for VDS - Using the same process as above, enter a DC source for VGS with the following:

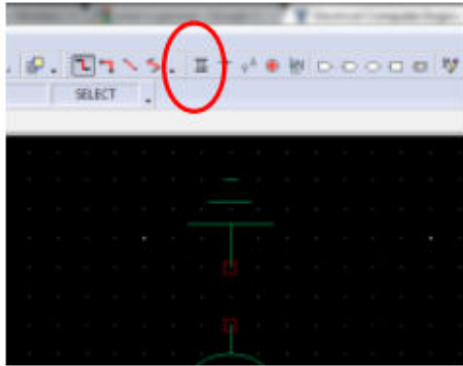
- MasterInterface: DC (this is the default but this is how you would change it to something else.
- Name: VDS_Source - V “VDS_param” Position the sources as in the following figure:



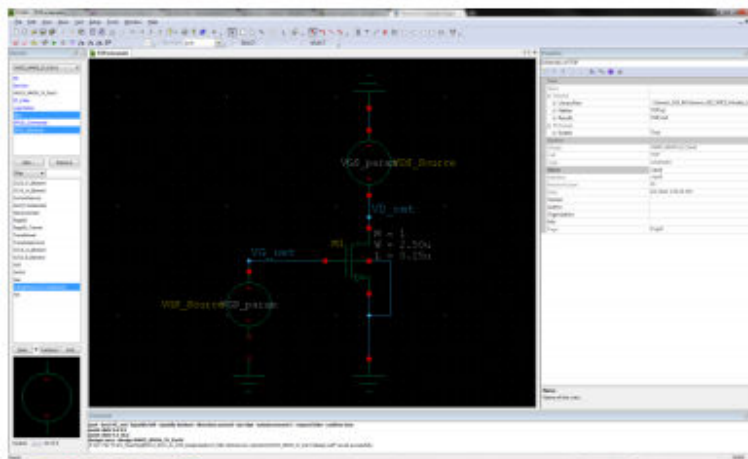
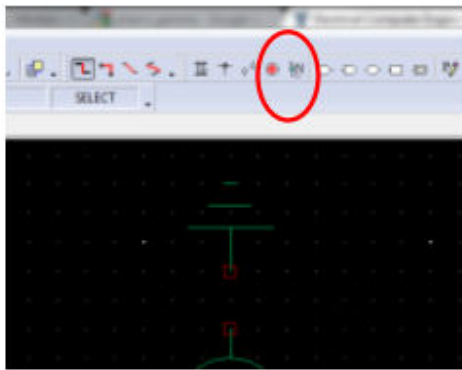
A note on zooming: - Hold down ALT-M to move a component. While holding these buttons down, click and drag the components. - To rotate, click on the device and click the [r] button. 8 d) Enter Grounds - Using the same process as above, enter 3 grounds from Misc:Gnd



e) Enter Wires - You can enter wires by clicking on the “wire” icon at the top



- Enter wires by clicking on a symbol node and then dragging. Enter corners by clicking once where you want to turn.
- You can label nets using the “Net Label” icon at the top



f) Enter a Current Probe to monitor IDS - Enter the SPICE_Commands:PrintCurrent component. This doesn't connect to anything. You just place it anywhere and then tell it what current to monitor in its properties dialog.

- In its properties dialog, setup: Terminal: D (this is the Drain of the NMOS) Device: MM1 (this is the name of the device. Notice that we called it M1, but S-edit automatically appends another M to the name. You will only see this once you run the Netlist. Analysis: DC (VERY IMPORTANT TO SELECT THIS!!!!)

Part 4: Setup the Parameters that will be used during the DC sweep analysis When we entered the VGS and VDS sources, we set their values to "VGS_param" and "VDS_param". We now need to setup these parameters. Using the pull down menus: - Setup

- SPICE Simulations - On the left, click on "Parameters"

- On the right, click on the "Add Parameters" button (it is in the upper right corner next to the red

X) Enter: Name: VGS_param Value: 1v - On the right, click on the "Add Parameters" button Enter: Name: VDS_param Value: 2.5v 10 We will overwrite these values during our sweep, but the parameters need to exist first.

Part 5: Setup the SPICE DC Sweep Analysis Using the pull down menus: - Setup

- SPICE Simulations - On the left, click on "DC Sweep Analysis"

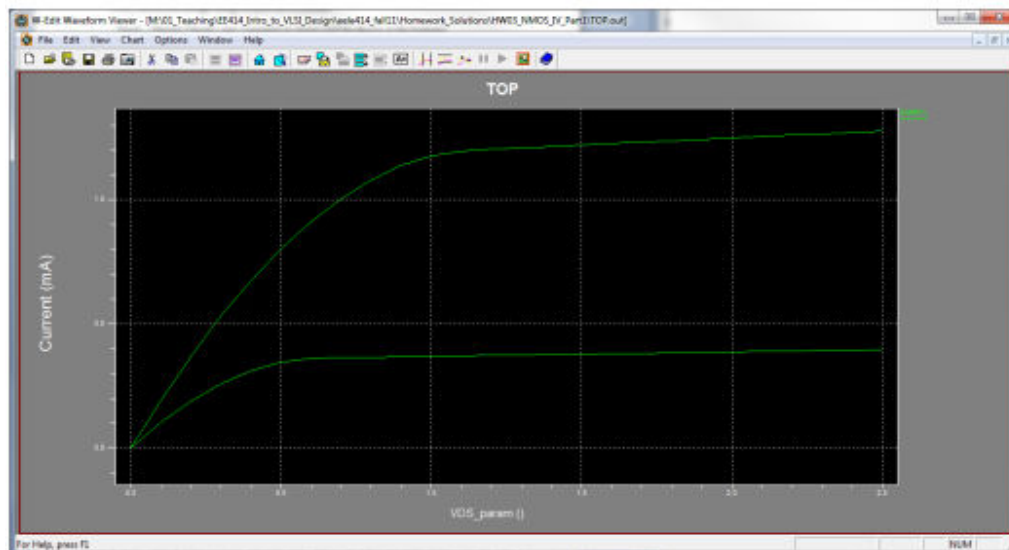
- On the right, enter the following for Source (this is what will be swept) Source or Parameter Name: VDS_param Start Value: 0 Stop Value: 2.5 Step: 0.1 Sweep Type: lin - On the right, enter the following for Source (this is what will be swept)

Source or Parameter Name: VGS_param Start Value: 1 Stop Value: 1.5 Step: 0.5 Sweep Type: lin

NOTE: The first parameter you setup in this dialog will be plotted on the independent axis. Part 6: Simulate the Design a) First, check you design using the pull down menus:

- Tools – Design Checks (any warnings or errors will be shown at the bottom) b) Simulate your design: - Click on the Green Arrow to start the simulator:

The T-Spice window will appear. If everything is OK, the waveform viewer will also appear. If everything worked, your waveforms should look like this:



c) View the Netlist: - In the T-spice window, right click on the file at the bottom and select “Show Netlist” This will bring up the TOP.sp Netlist that was created and used by the spice engine. This is a good place to look when you get errors. This is the text based description of what you entered in S-edit.

```

T-Spice - [TOP.sp]
File Edit View Simulation Table Setup Window Help

* SPICE export by: SEEST 12.41
* Export time: Wed Sep 07 15:22:58 2011
* Design: 1803_1805_IV_Part1
* Cell: TOP
* View: view0
* Export net: top-level netlist
* Export model: hierarchical
* Exclude .model: no
* Exclude .end: no
* Expand paths: yes
* Wrap lines: no
* Root path: M:\01_Teaching\EE414_Intro_to_VLSI_Design\ee414_fall11\Tanner_Projects\1803_1805_IV_Part1
* Exclude global paths: no
* Control property name: SPICE

***** Simulation Settings - General section *****
.lib "M:\01_Teaching\EE414_Intro_to_VLSI_Design\ee414_fall11\Tanner_Projects\Generic_GDS_Kic\Generic_GDS_SPICE_Models_Level1.lib"

***** Simulation Settings - Parameters and SPICE Options *****
.param VDS_param = 1
.param VDS_param = 2.5

***** Devices: SPICE.CHEN > 0 *****
M11 Vd_net Vd_net Gnd Gnd 1805 n=2.5u l=250u AS=2.25p FS=6.5u AD=2.25p RC=6.5u
VDS_source Vd_net Gnd DC VDS_param
.PRINT DC IS (M11)

***** Simulation Settings - Analysis section *****
.AC lin VDS_param 0 2.5 0.1 lin VDS_param 1 1.5 0.5

***** Simulation Settings - Additional SPICE commands *****
.end

Status Input File Output File Options Start/End Time Elapsed
Loaded M11 TOP.sp Top.sp September 07, 2011 15:22:26 00:0
Loaded M11 TOP.sp Top.sp September 07, 2011 15:22:26 00:0

```

d) View the Waveform: - If the windows viewer did NOT automatically appear, you can click on the file in the Tspice window and select “Show Waveform”.

3.3 Transient Analysis of a CMOS Inverter & Symbol Creation

Part 1: Start a New Design, Setup Libraries & Setup Simulation a) In this example, we will create a CMOS inverter and simulate its transient response. We will create an inverter design that contains a symbol and then instantiate it in another schematic to stimulate the circuit. Symbols are handled by adding another view to a design.

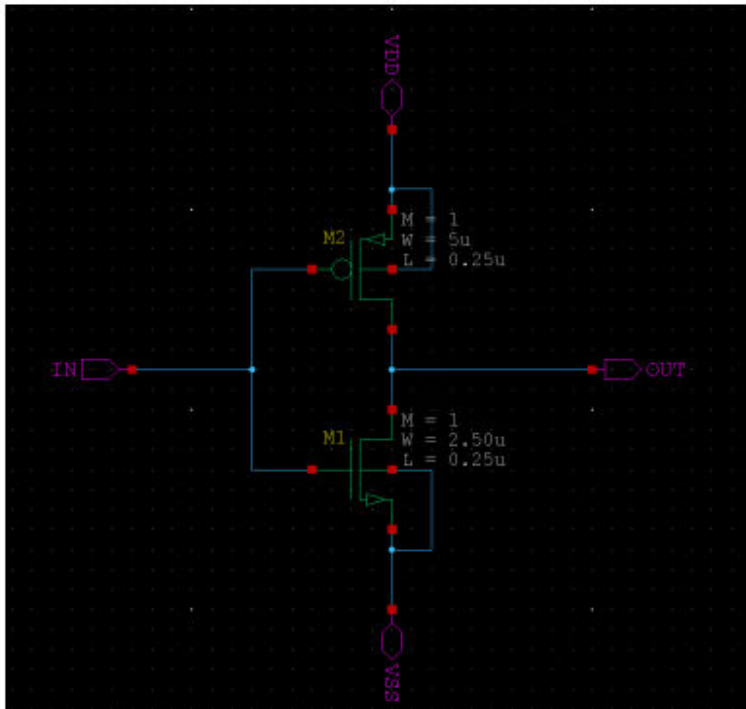
We will start by creating a design called “Inverter” and then create a schematic view. This schematic will contain an NMOS and PMOS wired as an inverter. We will add “Ports” for the Input, Output, VDD, and VSS. We will then add a symbol view to this design. The symbol will contain the inverter shape and the corresponding pins for Input, Output, VDD, and VSS.

We will then create a separate schematic called TOP that will be used to test the inverter. We will instantiate the inverter symbol in TOP. We only want to put items into the inverter design that can be fabricated. TOP will contain the ideal voltage sources to provide the input waveform, the power supplies, and a mock load.

In this way, when we go into physical design (i.e., layout), we only drive forward the isolated circuits.

- Start S-Edit - Create a new design called: “EELE414_VLSI_Fall2011\Tanner_Projects\HW04_INV_Transient_Part1” - Add the Tanner_Projects\Libraries\All\All.tanner library to the library list on the left

- Create a new Cell called “TOP” using the Pull Down Menus - Cell - New View Name = TOP View Type = schematic - Setup the simulation using the Pull Down Menus: - Setup – SPICE Simulation - Highlight the General Tab of the Setup SPICE window and set the following: SPICE File Name: \HW04_INV_Transient_Part1\TOP.sp Library Files: ..\Generic_025_Kit\Generic_025_SPICE_Models_Level1.lib Simulation Results File Name: File Name: \HW04_INV_Transient_Part1\TOP.out 13 - Check the “Transient/Fourier Analysis” box on the left and set the following: Stop Time = 2ns Maximum Time Step = 10ps - Click “OK” Part 2: Create the Inverter a) Create a new schematic view using the pull-down menus: - Cell - New View Name = Inverter View Type = schematic b) Enter the inverter schematic:

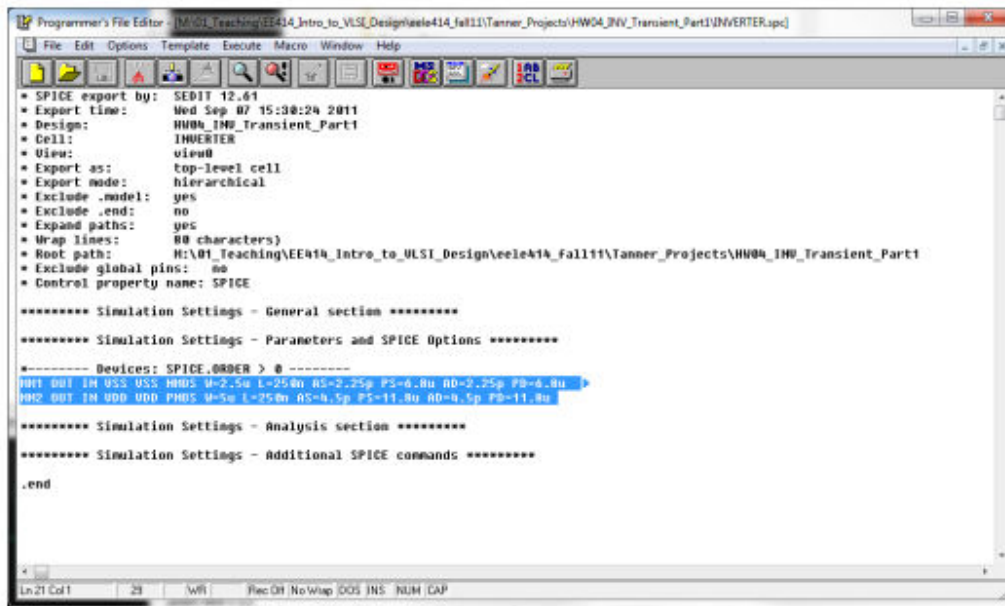


- Entering the NMOS: Name = M1 L = 0.25u W = 2.5u Model =NMOS 14
 - Entering the PMOS: Name = M2 L = 0.25u W = 5.0u Model =PMOS - Entering the Ports: Ports are entered using the icons on the top of the S-edit window.
- Enter the following: In Port: Name it “IN” Out Port: Name it “OUT” In/Out Port: Name it “VDD” In/Out Port: Name it “VSS”
- Wire up the Inverter Enter wire connections as shown in the previous figure.

c) Export a SPICE Netlist Exporting a SPICE Netlist is a good idea in order to verify that you have entered the schematic correctly. Also, this Netlist will be used later when performing a “Layout versus Schematic (LVS)” check.

We want to export a Netlist at the Inverter schematic cell level so that a Netlist of just the inverter exists for LVS. When we conduct the simulation of this inverter, we will create a TOP level schematic that will have a Netlist containing ideal voltage sources. This Netlist can’t be used for LVS since it contains components that won’t be fabricated. With the schematic open, use the pull down menus to perform:

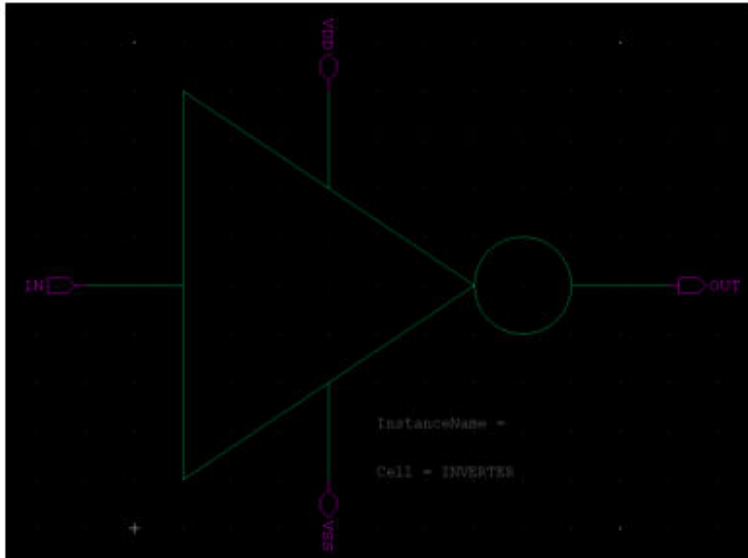
- File – Export – Export SPICE. - Browse to your design directory and give the file name “Inverter.spc”.
- Click “OK” If you open the Inverter.spc with a text editor, you will see the following:



d) Create the Inverter Symbol Symbols can either be created manually by creating a new symbol view or automatically by S-edit. We will use the automatic symbol generation. This will create a new symbol view from the schematic, create the ports for the symbol, and make a symbol shape. While the shape of the symbol is rarely what we ultimately want, it will do a lot of the work for us.

- With the schematic view open, use the pull down menus to create the symbol view:

Cell – Update Symbol A new window will come up with a square symbol and 4 ports with the same names you entered in the Inverter:schematic view (i.e., IN, OUT, VDD, VSS). You should edit the shapes until you have created a symbol that looks like an inverter:



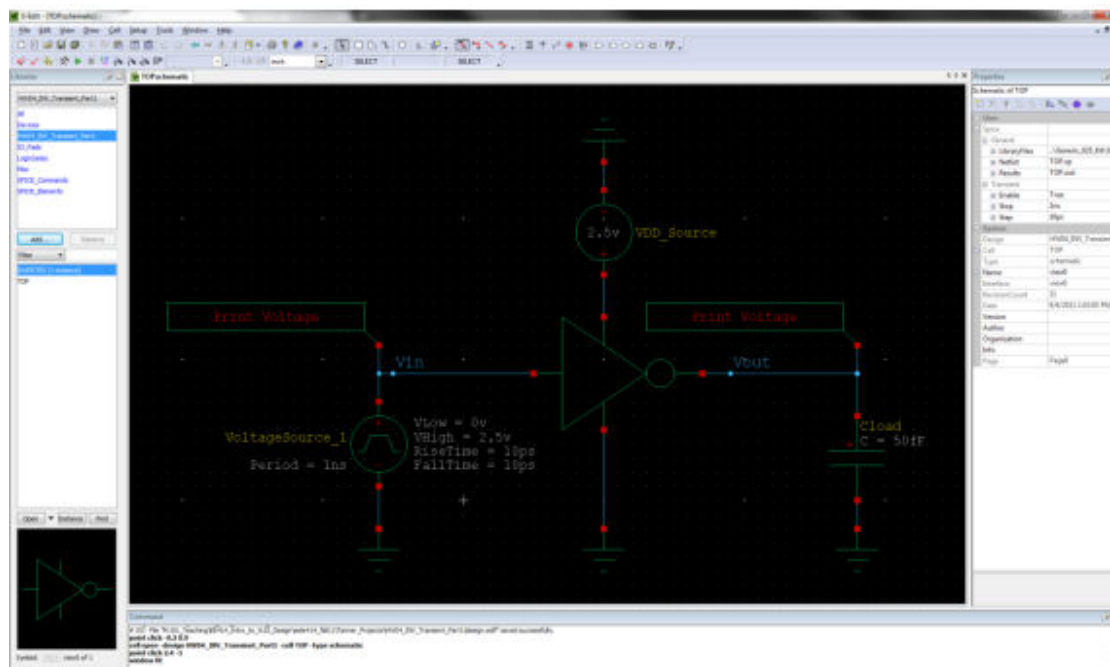
A note on drawing: The “Path” icon will put you into a mode where you can draw lines that are not wires. The “Circle” icon will allow you to enter the inversion bubble. The ports can be moved by holding down “alt-m” The ports can be rotated by selecting and pressing the “r” button Remember to save.

Part 3: Create the TOP schematic to test the Inverter

a) Instantiate the Inverter in the TOP schematic Open the TOP schematic view using the pull-down menus:

- Cell – Open View: Cell Name: TOP View Type: schematic In the library windows on the left of the window, highlight your “HW04_INV_Transient_Part1” library. In the lower left window, you will see your two Cells “TOP” and “Inverter”.

- Click on “Inverter” and you will see your symbol show up in the symbol viewer. - Click on the “Instance” button and place your symbol in the TOP schematic. 17 b) Enter the following circuit in order to power and stimulate your inverter:



- Enter the Pulse Voltage Source.

All voltage sources are the same component in the SPICE_Elements library. The default is DC, but this can be changed to any other type of source in the properties dialog.

Name = Vin_SourceMasterInterface = Pulse Period = 1ns PulseWidth = 0.5ns VHigh = 2.5v VLow = 0v RiseTime = 10ps FallTime = 10ps - Enter a Load capacitor from the Devices library. Name = Cload C = 50fF - Enter a DC Source for VDD Name = VDD_SourceMasterInterface = DC V = 2.5v

- Enter the grounds from the Misc library 18

- Enter wire connections and name them Vin and Vout

- Enter a voltage probe for both Vin and Vout Part 4: Simulate the Design

a) First, check you design using the pull down menus: - Tools – Design Checks (any warnings or errors will be shown at the bottom)

b) Simulate your design: - Click on the Green Arrow to start the simulator: The T-Spice window will appear. If everything is OK, the waveform viewer will also appear. If everything worked, your waveforms should look like this:

